



Fatima Jinnah Women University

Opening Portals of Excellence Through Higher Education

Open-Ended Lab Report

Terraform & Ansible – Frontend Backend High Availability Architecture

Student Name:

Manahil Habib

Registration No:

2023-BSE-042

Course:

Cloud Computing

Lab Type:

Open Ended Lab

1. Introduction

This report presents the complete implementation of an open-ended cloud computing lab using Terraform and Ansible.

The goal of this lab is to provision infrastructure automatically and configure a highly available frontend-backend web architecture.

2. GitHub Repository Initialization

A dedicated GitHub repository was created to maintain version control and ensure clean separation of this lab from previous assignments

```
@manahilhabib031 →/workspaces/cc-manahilhabib-042 (main) $ mkdir -p LabProject_FrontendBackend
@manahilhabib031 →/workspaces/cc-manahilhabib-042 (main) $ cd LabProject_FrontendBackend
```

3. GitHub Codespaces Environment Setup

GitHub Codespaces was used as the development environment to provide a consistent Linux- based setup.

```
C:\Users\HABIB TARIQ>gh codespace list
NAME                               DISPLAY_NAME                     REPOSITORY                      BRANCH  STATE      CREATED AT
expert-space-pancake-pj5r595...   expert space pancake            manahilhabib031/Lab9            main*   Shutdown   about 19 days ago
didactic-space-doodle-6vv7xq...   didactic space doodle          manahilhabib031/Lab12           main*   Shutdown   about 9 days ago
expert-garbanzo-wrq7qxqx66r4...   expert garbanzo                 manahilhabib031/Lab13           main*   Shutdown   about 8 days ago
legendary-lamp-r45g5w5w95v4h...   legendary lamp                  manahilhabib031/terrafo...      main*   Shutdown   about 5 days ago
crispy-space-tribble-jjp5p9p...   crispy space tribble           manahilhabib031/labexam         main*   Shutdown   about 9 hours ago
bookish-carnival-jjp5p9p96pj...   bookish carnival               manahilhabib031/cc-mana...      main*   Available  about 6 hours ago

C:\Users\HABIB TARIQ>ssh -c bookish-carnival-jjp5p9p96pjg27gx
Unknown cipher type 'bookish-carnival-jjp5p9p96pjg27gx'

C:\Users\HABIB TARIQ>gh codespace ssh -c bookish-carnival-jjp5p9p96pjg27gx
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@manahilhabib031 →/workspaces/cc-manahilhabib-042 (main) $
```

4. Tool Installation & Verification

Required tools including Terraform, AWS CLI, Python, and Ansible were installed and verified before implementation.

```
@manahilhabib031 →/workspaces/cc-manahilhabib-042 (main) $ terraform version
Terraform v1.14.3
on linux_amd64
@manahilhabib031 →/workspaces/cc-manahilhabib-042 (main) $ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/codespace/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/codespace/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Nov 6 2025, 13:44:16) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.6
  libyaml = True
@manahilhabib031 →/workspaces/cc-manahilhabib-042 (main) $ aws --version
```

```

@manahilhabib031 → /workspaces/cc-manahilhabib-042 (main) $ /workspaces/cc-manahilha
bib-042/LabProject_FrontendBackend
-bash: /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend: Is a directory
@manahilhabib031 → /workspaces/cc-manahilhabib-042 (main) $ touch main.tf variables.
tf outputs.tf locals.tf terraform.tfvars
@manahilhabib031 → /workspaces/cc-manahilhabib-042 (main) $ ls
Assignmet1      Lab1      largefile2.zip
Assignmts      Lab1a     largefile3.zip
'Lab 07'       Lab2      locals.tf
'Lab 08'       Lab3      main
'Lab 09'       Lab4      main.tf
'Lab 10'       Lab5      outputs.tf
'Lab 11'       LabProject_FrontendBackend temp.txt
'Lab 12'       README.md terraform.tfvars
'Lab 13 cc.pdf' assignment-02 textfiles
'Lab 14'       'lab 1 CC.docx' variables.tf
'Lab 6'        'lac4 cc.docx'
'Lab exam'     largefile1.zip
@manahilhabib031 → /workspaces/cc-manahilhabib-042 (main) $

```

```

@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /home/codespace/.ssh/id_ed25519
Your public key has been saved in /home/codespace/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:lweLhs9wgUTRDxqUo7vSv3ws57bnTw4qVNwcVLMODOU codespace@codespaces-593764
The key's randomart image is:
+--[ED25519 256]--+
|      .+oo+.oo.o |
|      . ++ .+. o |
|      o.oo.oE.. |
|      .o oo+oo |
|      o.S.+ . . |
|      .*. . . |
|      . oo. . . |
|      . o.o =..+ |
|      . .+O++..o |
+-----[SHA256]-----+
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ ls ~/.ssh
authorized_keys  id_ed25519  id_ed25519.pub
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ |

```

5. Final Project Directory Structure

The final directory structure follows Terraform module-based design and Ansible role-based configuration.

```

@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend
-bash: /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend: Is a directory
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ cd /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ mkdir -p ansible/{playbooks,inventory,roles}
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ mkdir -p ansible/roles/{backend,frontend}/{tasks,handlers,templates}
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ touch ansible/ansible.cfg
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ touch ansible/playbooks/site.yaml
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ touch ansible/inventory/hosts
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)
$ tree ansible
ansible
├── ansible.cfg
├── inventory
│   └── hosts
├── playbooks
│   └── site.yaml
└── roles
    ├── backend
    │   ├── handlers
    │   ├── tasks
    │   └── templates
    └── frontend
        ├── handlers
        ├── tasks
        └── templates
12 directories, 3 files
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main)

```

6. Terraform Configuration

6.1 Provider & Variables Definition

Terraform provider configuration specifies the AWS region and credentials. Variables are used for reusability and flexibility.

```

@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $ cat variables.tf
variable "vpc_cidr" { default = "10.0.0.0/16" }
variable "subnet_cidr" { default = "10.0.1.0/24" }
variable "env_prefix" { default = "lab-project" }
variable "instance_type" { default = "t2.micro" }
variable "public_key_path" { default = "~/.ssh/id_ed25519.pub" }
variable "private_key_path" { default = "~/.ssh/id_ed25519" }
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $

```

6.2 Local Values & IP Restriction

Local values dynamically retrieve the user's public IP to restrict SSH access securely.

```
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $ cat locals.tf
data "http" "my_ip" {
  url = "https://icanhazip.com"
}

locals {
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"
}
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $ |
```

6.3 Networking (VPC, Subnet, Routing)

A VPC, public subnet, internet gateway, and route table are provisioned to allow internet access.

```
GNU nano 7.2 main.tf
provider "aws" {
  region = var.region

data "aws_ami" "amazon_linux" {
  most_recent = true
  filter {
    name     = "name"
    values   = ["amzn2-ami-hvm-*x86_64-gp2"]
  }
  owners = ["amazon"]

module "network" {
  source = "./modules/subnet"

  vpc_cidr_block      = var.vpc_cidr_block
  subnet_cidr_block   = var.subnet_cidr_block
  availability_zone    = var.availability_zone
  env_prefix           = var.env_prefix

resource "aws_security_group" "web_sg" {
  name     = "${var.env_prefix}-sg"
  vpc_id   = module.network.vpc_id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [local.my_ip]
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
  }
}
```

6.4 EC2 Frontend & Backend Provisioning

One frontend EC2 instance and three backend EC2 instances are created using reusable Terraform modules.

```
GNU nano 7.2 main.tf

resource "aws_key_pair" "key" {
  key_name   = "${var.env_prefix}-key"
  public_key = file(var.public_key_path)
}

module "frontend" {
  source = "./modules/webserver"

  ami_id           = data.aws_ami.amazon_linux.id
  instance_type    = var.instance_type
  subnet_id       = module.network.subnet_id
  security_group_ids = [aws_security_group.web_sg.id]
  key_name        = aws_key_pair.key.key_name
  name            = "${var.env_prefix}-frontend"
  count           = 1
}

module "backend" {
  source = "./modules/webserver"

  ami_id           = data.aws_ami.amazon_linux.id
  instance_type    = var.instance_type
  subnet_id       = module.network.subnet_id
  security_group_ids = [aws_security_group.web_sg.id]
  key_name        = aws_key_pair.key.key_name
  name            = "${var.env_prefix}-backend"
  count           = 3
}

resource "null_resource" "ansible_run" {
  depends_on = [
    module.frontend,
    module.backend
  ]
}
```

6.5 Terraform Outputs

Terraform outputs expose public and private IP addresses required for Ansible inventory and verification.

```
Command Prompt - gh codespace ssh -c fictional-pancake-wrq7vwqxg6rhq6g
GNU nano 7.2 outputs.tf
output "frontend_public_ip" {
  value = aws_instance.frontend.public_ip
}

output "backend_public_ips" {
  value = [for b in aws_instance.backend : b.public_ip]
}

output "backend_private_ips" {
  value = [for b in aws_instance.backend : b.private_ip]
}
```

Outputs:

```
backend_private_ips = [
  "10.0.1.217",
  "10.0.1.168",
  "10.0.1.150",
]
backend_public_ips = [
  "44.199.199.14",
  "100.31.203.179",
  "98.86.167.23",
]
frontend_public_ip = "98.82.17.19"
```

6.6 Terraform Modules Implementation

Terraform modules were used to improve code reusability, readability, and separation of concerns. Instead of defining all resources in a single file, networking and compute logic were encapsulated into independent modules.

This approach aligns with infrastructure-as-code best practices and simplifies future scaling.

6.6.1 Subnet Module (Networking Layer)

The subnet module is responsible for provisioning the core networking components required for the project. It creates the Virtual Private Cloud (VPC), public subnet, internet gateway, route table, and route table association.

This ensures that all EC2 instances are deployed in a properly configured public network with internet access

```
GNU nano 7.2 main.tf
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main_vpc.id
  tags = merge(local.common_tags, {
    Name = "${var.project_name}-igw"
  })
}
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main_vpc.id
  tags = merge(local.common_tags, {
    Name = "${var.project_name}-public-rt"
  })
}
resource "aws_route" "default_route" {
  route_table_id = aws_route_table.public_rt.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id = aws_internet_gateway.igw.id
}
resource "aws_route_table_association" "public_assoc" {
  count = length(aws_subnet.public)
  subnet_id = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public_rt.id
}
resource "aws_instance" "frontend" {
  ami = data.aws_ami.amazon_linux.id # Amazon Linux 2 (ME Central)
  instance_type = "t3.micro"
  subnet_id = aws_subnet.public[0].id
  key_name = aws_key_pair.deployer.key_name
  vpc_security_group_ids = [aws_security_group.web_sg.id]
  associate_public_ip_address = true

  tags = merge(local.common_tags, {
    Name = "${var.project_name}-frontend"
    Role = "frontend"
  })
}
```

```
Command Prompt - gh codespace ssh -c fictional-pancake-wrq7vwqxg6rhq6g
GNU nano 7.2 variables.tf
variable "project_name" {
  description = "Project name prefix"
  type = string
  default = "frontend-backend-lab"
}
variable "region" {
  description = "AWS region"
  type = string
  default = "me-central-1"
}
variable "vpc_cidr" {
  description = "CIDR block for VPC"
  type = string
  default = "10.0.0.0/16"
}
variable "public_subnet_cidrs" {
  description = "Public subnet CIDRs"
  type = list(string)
  default = [
    "10.0.1.0/24",
    "10.0.2.0/24"
  ]
}
variable "instance_type" {
  description = "EC2 instance type"
  type = string
  default = "t3.micro"
}
```



```
GNU nano 7.2 modules/subnet/variables.tf *
variable "vpc_cidr_block" {
  type = string
}

variable "subnet_cidr_block" {
  type = string
}

variable "availability_zone" {
  type = string
}

variable "env_prefix" {
  type = string
}
-
```

```
GNU nano 7.2 modules/subnet/outputs.tf
output "vpc_id" {
  value = aws_vpc.this.id
}

output "subnet_id" {
  value = aws_subnet.this.id
}
```

6.6.2 Webserver Module (EC2 Abstraction Layer)

The webserver module abstracts EC2 instance creation into a reusable component.

It is used to provision both frontend and backend instances by passing different parameters such as instance count and naming prefix.

This modular design avoids duplication and ensures consistent EC2 configuration.

7. Ansible Configuration:

7.1 Ansible Configuration File

The ansible.cfg file disables host key checking and specifies Python interpreter settings.

```
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $ cat ansible/ansible.cfg
[defaults]
host_key_checking = False
inventory = ./inventory/hosts.ini
remote_user = ec2-user
# ADD THIS LINE: Tell Ansible where to find the 'frontend' and 'backend' roles
roles_path = roles
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $
```

7.2 Inventory File

The inventory file groups frontend and backend servers to enable role-based execution.

```
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $ cat ansible/inventory/hosts
[frontend]
44.213.104.4

[backends]
3.239.30.159
3.239.110.214
34.200.234.94

[all:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $
```

7.3 Backend Role (Apache HTTPD)

The backend role installs Apache HTTPD and deploys unique index pages on each backend server.

```
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBacke
nd (main) $ cat ansible/roles/backend/tasks/main.yml
---
- name: Install Apache HTTPD
  yum:
    name: httpd
    state: present
    update_cache: yes

- name: Enable and Start HTTPD
  service:
    name: httpd
    state: started
    enabled: yes

- name: Deploy Custom Index Page
  template:
    src: backend_index.html.j2
    dest: /var/www/html/index.html
    mode: '0644'
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBacke
nd (main) $
```

7.4 Frontend Role (Nginx Load Balancer)

The frontend role installs Nginx and configures it as a reverse proxy with two primary and one backup backend.

```
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBacke
nd (main) $ cat ansible/roles/frontend/tasks/main.yml
---
- name: Install Nginx
  yum:
    name: nginx
    state: present
    update_cache: yes

- name: Deploy Nginx Config with Load Balancing
  template:
    src: nginx_frontend.conf.j2
    dest: /etc/nginx/nginx.conf
    notify: Restart Nginx

- name: Enable and Start Nginx
  service:
    name: nginx
    state: started
    enabled: yes
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBacke
nd (main) $
```

```
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBacke
nd (main) $ cat ansible/roles/frontend/handlers/main.yml
---
- name: Restart Nginx
  service:
    name: nginx
    state: restarted
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBacke
nd (main) $
```

```
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBacke
nd (main) $ cat ansible/roles/frontend/templates/nginx_frontend.conf.j2
access_log /var/log/nginx/access.log main;

# LOAD BALANCER CONFIGURATION
upstream backend_servers {
    # Primary Backends (Round Robin)
    server {{ backend1_private_ip }}:80;
    server {{ backend2_private_ip }}:80;

    # Backup Backend (Only used if others fail)
    server {{ backup_backend_private_ip }}:80 backup;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backend_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

7.5 Main Playbook (site.yaml)

The main playbook applies backend configuration first, followed by frontend setup.

```
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $ cat ansible/playbooks/site.yaml
---
# 1. Configure Backends first to ensure IPs are gathered
- name: Configure Backend Servers
  hosts: backends
  become: true
  gather_facts: true
  roles:
    - backend

# 2. Configure Frontend with IPs from the 'backends' group
- name: Configure Frontend Load Balancer
  hosts: frontend
  become: true
  gather_facts: true
  vars:
    # Dynamically grab Private IPs from the inventory groups
    backend1_private_ip: "{{ hostvars[groups['backends'][0]]['ansible_default_ipv4']['address'] }}"
    backend2_private_ip: "{{ hostvars[groups['backends'][1]]['ansible_default_ipv4']['address'] }}"
    backup_backend_private_ip: "{{ hostvars[groups['backends'][2]]['ansible_default_ipv4']['address'] }}"
  roles:
    - frontend
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $
```

8. Terraform Apply & Automation Execution

Terraform apply provisions all resources and automatically triggers Ansible using a null_resource.

```
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBackend/ansible (main) $ ansible-playbook -i inventory/hosts.ini playbooks/site.yaml

PLAY [Configure Frontend Load Balancer] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host 98.82.17.19 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for more information.
ok: [98.82.17.19]

TASK [frontend : Install Nginx] *****
ok: [98.82.17.19]

TASK [frontend : Deploy Nginx Config with Load Balancing] *****
ok: [98.82.17.19]

TASK [frontend : Enable and Start Nginx] *****
ok: [98.82.17.19]

PLAY RECAP *****
100.31.203.179      : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
44.199.199.14      : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
98.82.17.19        : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
98.86.167.23       : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

```
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $ terraform apply -auto-approve
"global" settings to all configurations in your organization. To reduce the verbosity of
these warnings, use the -compact-warnings option.

Warning: Values for undeclared variables

In addition to the other similar warnings shown, 3 other variable(s) defined without
being declared.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

backend_private_ips = [
  "10.0.1.217",
  "10.0.1.168",
  "10.0.1.150",
]
backend_public_ips = [
  "44.199.199.14",
  "100.31.203.179",
  "98.86.167.23",
]
frontend_public_ip = "98.82.17.19"
```

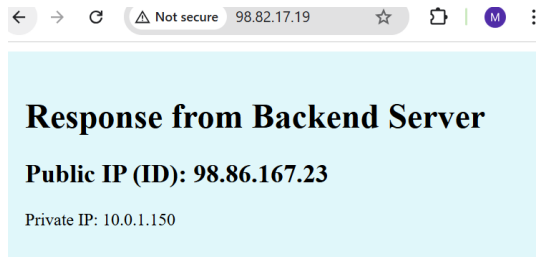
9. Backend Server Verification

The following unlabeled screenshots show direct access to backend servers using their public IPs.



Backend Server

10. Frontend Load Balancer Verification



Frontend Response

```
@manahilhabib031 → /workspaces/cc-manahilhabib-042/LabProject_FrontendBackend (main) $ for
i in {1..5}; do curl http://98.82.17.19; echo "\n---"; sleep 1; done
<!DOCTYPE html>
<html>
<head><title>Backend 98.86.167.23</title></head>
<body>
  <div style="background-color: #e0f7fa; padding: 20px;">
    <h1>Response from Backend Server</h1>
    <h2>Public IP (ID): 98.86.167.23</h2>
    <p>Private IP: 10.0.1.150</p>
  </div>
</body>
</html>
\n---
<!DOCTYPE html>
<html>
<head><title>Backend 98.86.167.23</title></head>
<body>
  <div style="background-color: #e0f7fa; padding: 20px;">
    <h1>Response from Backend Server</h1>
    <h2>Public IP (ID): 98.86.167.23</h2>
    <p>Private IP: 10.0.1.150</p>
  </div>
</body>
</html>
\n---
<!DOCTYPE html>
<html>
<head><title>Backend 98.86.167.23</title></head>
<body>
  <div style="background-color: #e0f7fa; padding: 20px;">
    <h1>Response from Backend Server</h1>
    <h2>Public IP (ID): 98.86.167.23</h2>
    <p>Private IP: 10.0.1.150</p>
  </div>
</body>
</html>
\n---
<!DOCTYPE html>
<html>
<head><title>Backend 98.86.167.23</title></head>
<body>
  <div style="background-color: #e0f7fa; padding: 20px;">
    <h1>Response from Backend Server</h1>
    <h2>Public IP (ID): 98.86.167.23</h2>
    <p>Private IP: 10.0.1.150</p>
  </div>
</body>
</html>
\n---
<!DOCTYPE html>
<html>
<head><title>Backend 98.86.167.23</title></head>
<body>
  <div style="background-color: #e0f7fa; padding: 20px;">
    <h1>Response from Backend Server</h1>
    <h2>Public IP (ID): 98.86.167.23</h2>
    <p>Private IP: 10.0.1.150</p>
  </div>
</body>
</html>
\n---
<!DOCTYPE html>
<html>
<head><title>Backend 98.86.167.23</title></head>
<body>
  <div style="background-color: #e0f7fa; padding: 20px;">
    <h1>Response from Backend Server</h1>
    <h2>Public IP (ID): 98.86.167.23</h2>
    <p>Private IP: 10.0.1.150</p>
  </div>
</body>
</html>
\n---
<!DOCTYPE html>
<html>
```

11. Failover Testing (Backup Backend)

```
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBackend/ansible (
main) $ curl -s http://98.82.17.19 | grep "Public IP"
      <h2>Public IP (ID): 100.31.203.179</h2>
@manahilhabib031 →/workspaces/cc-manahilhabib-042/LabProject_FrontendBackend/ansible (
main) $
```

Backup Backend Response

12. Conclusion

This lab successfully demonstrates automated infrastructure provisioning and configuration management. The architecture meets all requirements of high availability, modularity, and automation.

